

✦ 每週只需不到1美元即可無限制存取 Medium 的最佳內容。 [成為會員](#)



機器學習很有趣！第四部分：基於深度學習的現代人臉辨識



亞當蓋特吉

跟隨

閱讀需 13 分鐘 · 2016 年 7 月 24 日



32 千



199

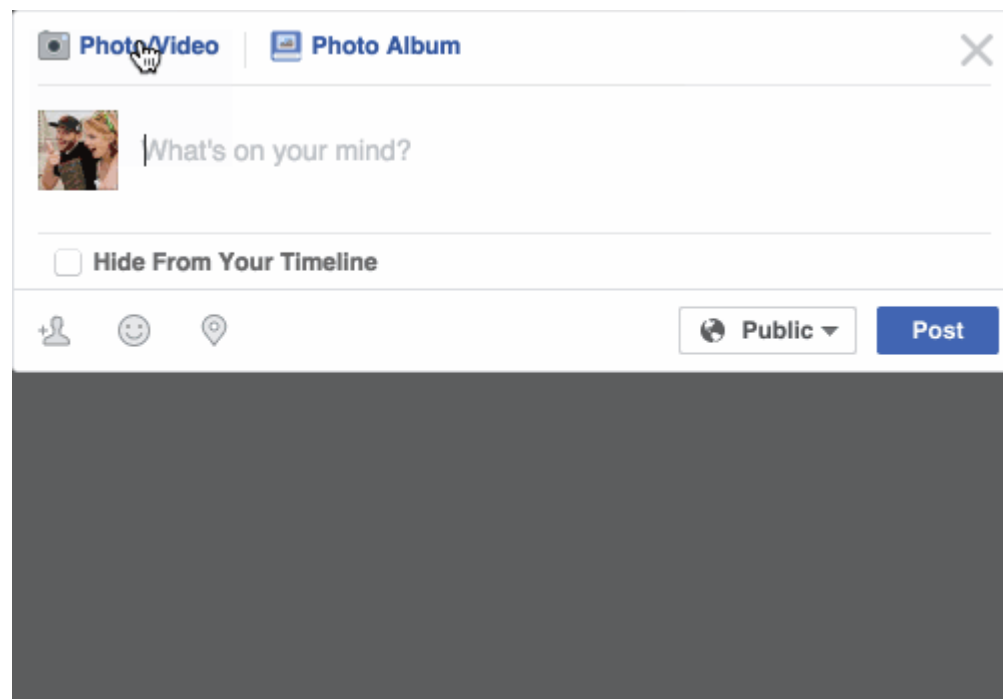


更新：本文是系列文章的一部分。看完整系列：[第 1 部分](#)、[第 2 部分](#)、[第 3 部分](#)、[第 4 部分](#)、[第 5 部分](#)、[第 6 部分](#)、[第 7 部分](#)和[第 8 部分](#)！您也可以使用[國語](#)、[Русский](#)、[한국어](#)、[Português](#)、[Tiếng Việt](#)、[Español](#)或[Italiano](#)閱讀這篇文章。

重大更新：[我根據這些文章寫了一本新書](#)！它不僅擴展和更新了我所有的文章，而且還包含大量全新的內容和大量的實際編碼項目。[現在就去看看吧](#)！

3

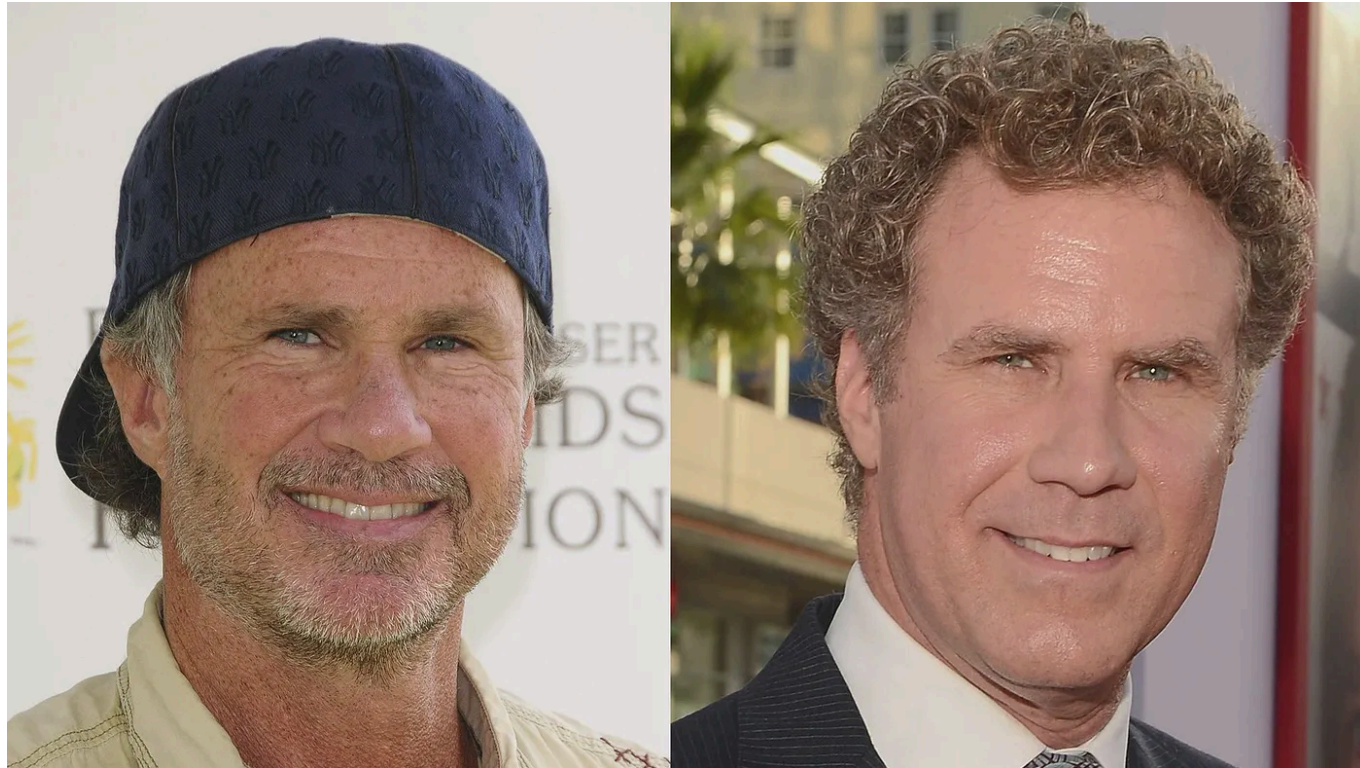
您是否注意到 Facebook 已經發展出一種神奇的能力，可以透過照片識別您的朋友？過去，Facebook 會要求您透過點擊照片並輸入其姓名來標記您的好友。現在，只要您上傳照片，Facebook 就會為您標記所有人像魔術一樣：



Facebook 會自動在您的照片中標記您先前標記過的人。我不確定這是否有用或令人毛骨悚然！

這項技術被稱為人臉辨識。只要標記幾次，Facebook 的演算法就能辨識你的朋友的人臉。這是一項非常了不起的技術——Facebook 可以辨識人臉準確率98%這幾乎和人類所能做到的一樣好！

讓我們了解現代人臉辨識的工作原理！但光是認出你的朋友就太容易了。我們可以將這項技術推向極限，以解決更具挑戰性的問題——區分威爾法瑞爾（著名演員）和查德史密斯（著名搖滾音樂家）！



其中一位就是威爾法瑞爾。另一位是查德史密斯。我發誓他們是不同的人！

如何利用機器學習解決非常複雜的問題

到目前為止，在第1、2和3部分中，我們已經使用機器學習來解決只有一個步驟的孤立問題 - 估算房價、根據現有數據生成新數據以及判斷圖像是否包含某個對象。所有這些問題都可以透過選擇一種機器學習演算法、輸入資料並獲得結果來解決。

但人臉辨識其實是一系列相關問題：

1. 首先，看一張圖片，找出其中的所有臉孔
2. 其次，專注於每一張臉，並能夠理解，即使臉轉向奇怪的方向或光線不好，它仍然是同一個人。
3. 第三，能夠找出臉部獨特的特徵，以便與其他人區分開來，例如眼睛有多大，臉有多長等等。
4. 最後，將該人臉的獨特特徵與您已經認識的所有人進行比較，以確定此人的名字。

身為人類，你的大腦天生就能自動、即時地完成所有這些事情。事實上，人類非常擅長辨識人臉，最終會在日常物品中看到人臉：



電腦還不具備這種高階概括能力（至少現在還不具備...），所以我們必須教導它們如何分別完成這個過程中的每個步驟。

我們需要建立一個管道，分別解決人臉辨識的每個步驟，並將目前步驟的結果傳遞到下一步。換句話說，我們將把幾種機器學習演算法連結在一起：



人臉偵測的基本流程是如何運作的

人臉辨識－逐步指南

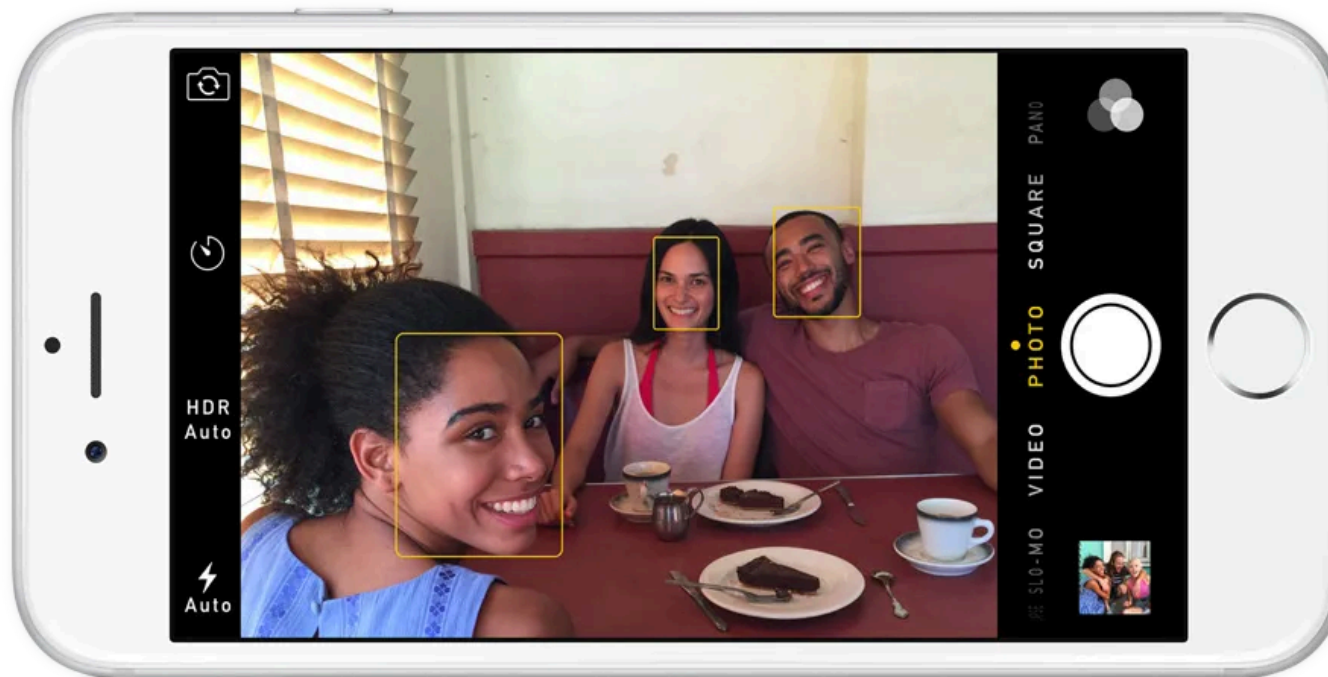
讓我們一步一步解決這個問題。對於每個步驟，我們將學習不同的機器學習演算法。我不會完整解釋每一個演算法以防止它變成一本書，但您將了解每個演算法背後的主要思想，並學習如何使用 OpenFace 和 dlib 在 Python 中建立自己的臉部辨識系統。

步驟 1：找到所有面孔

3

我們流程的第一步是**人臉偵測**。顯然，我們需要先找到照片中的臉孔，然後才能嘗試區分它們！

如果您在過去 10 年內使用過任何相機，您可能已經看過人臉偵測的實際應用：



人臉偵測是相機的一項很棒的功能。當相機可以自動辨識人臉時，它可以確保在拍照之前所有人臉都清晰對焦。但我們將使用它來達到不同的目的——找到我們想要傳遞到管道下一步的圖像區域。

3

21 世紀初，保羅·維奧拉 (Paul Viola) 和邁克爾·瓊斯 (Michael Jones) 發明了一種可以在廉價相機上運行且速度足夠快的人臉檢測方法，從此人臉檢測開始成

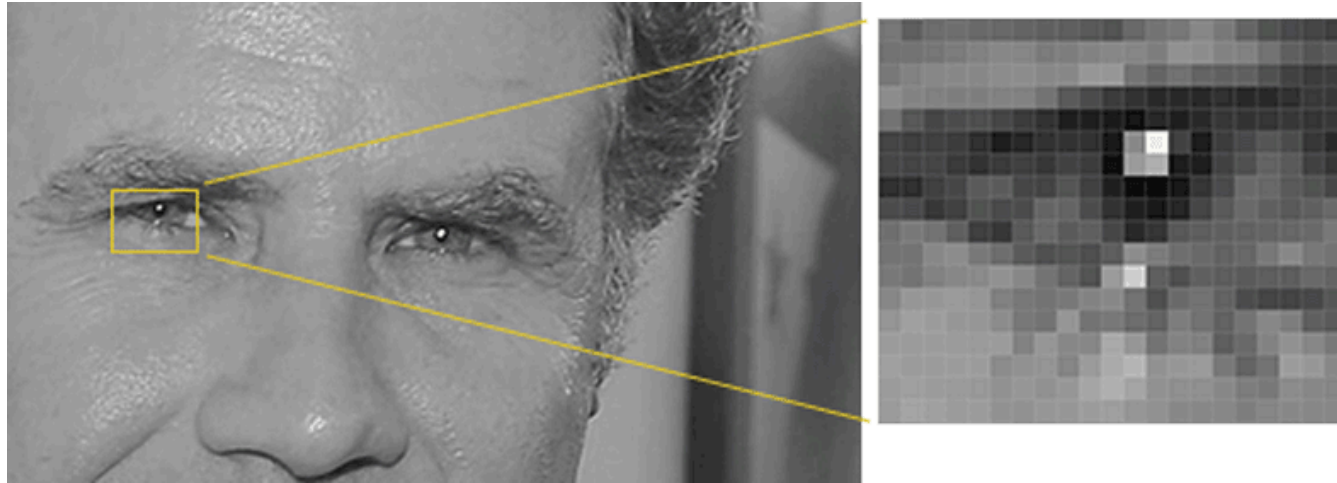
為主流。然而，現在已經有了更可靠的解決方案。我們將使用 2005年發明的一種方法 稱為方向梯度直方圖——或者簡稱 **霍格** 簡稱。

為了在圖像中找到人臉，我們首先將圖像變成黑白色，因為我們不需要顏色資料來尋找人臉：

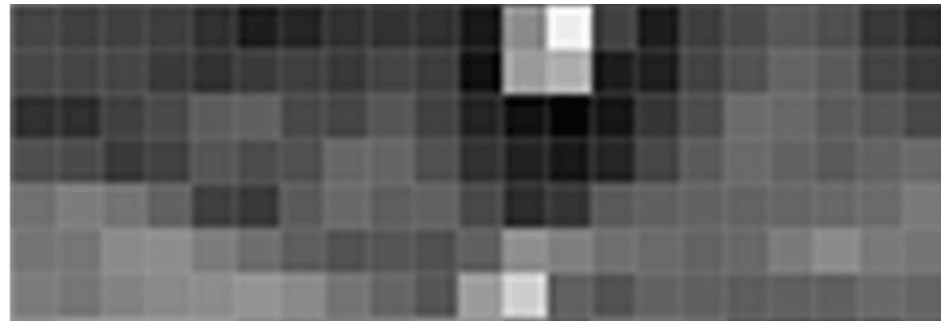


然後我們將逐一查看圖像中的每個像素。對於每一個像素，我們都想查看它周圍的像素：

3



我們的目標是弄清楚當前像素與周圍的像素相比有多暗。然後我們要畫一個箭頭來表示影像變暗的方向：



只看這個像素和與其接觸的像素，影像的右上角會變暗。

3

如果對影像中的每個像素重複此過程，最終每個像素都會被箭頭取代。這些箭頭稱為漸變，它們顯示了整個影像從亮到暗的流動：



這看起來似乎是一件隨機的事情，但是用漸層代替像素確實有很好的理由。如果我們直接分析像素，同一個人的非常暗的圖像和非常亮的圖像將具有完全不同的像素值。但僅考慮方向隨著亮度的變化，非常暗的圖像和非常明亮的圖像最終都會呈現完全相同的表現。這使得問題更容易解決！

但是保存每個像素的漸變會為我們帶來太多細節。我們最後只見樹木不見森林。如果我們能夠在更高層次上看到明暗的基本流動，那就更好了，這樣我們就能看到影像的基本圖案。

為此，我們將影像分成每個 16x16 像素的小方塊。在每個方格中，我們將計算每個主要方向上有多少個漸層點（有多少個指向上方、有多少個指向右上方、

有多少個指向右方等等...)。然後我們將用最強的箭頭方向來取代影像中的那個方塊。

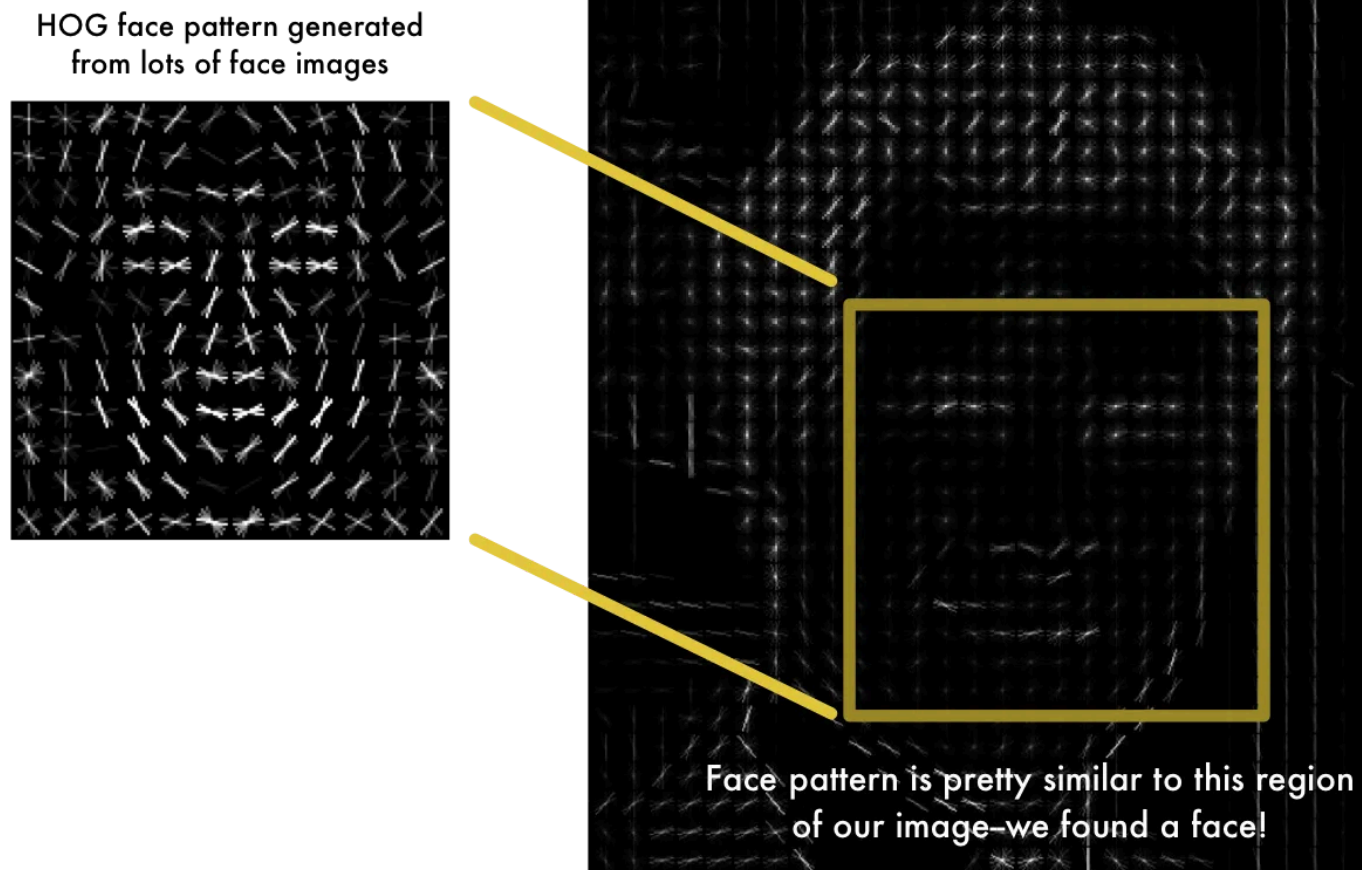
最終的結果是，我們將原始影像轉換成一個非常簡單的表示，以簡單的方式捕捉臉部的基本結構：



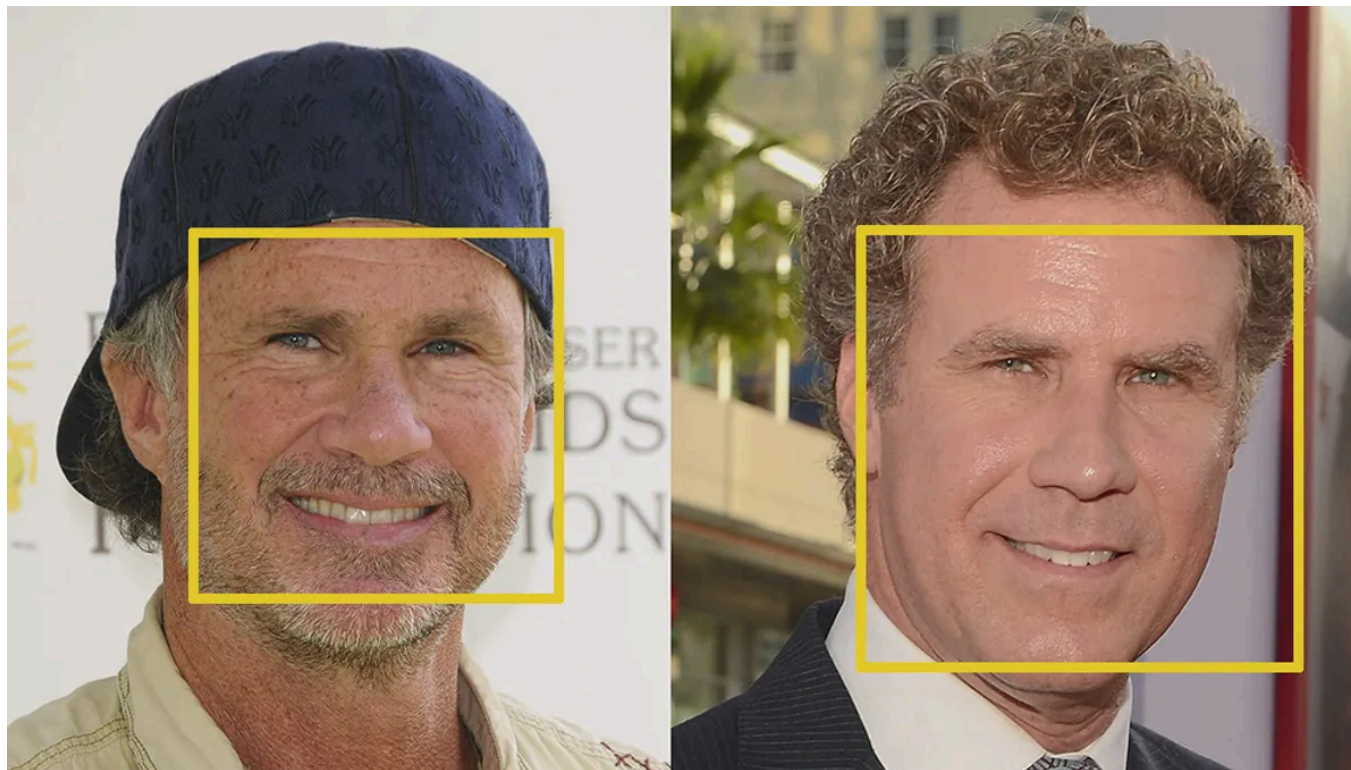
原始影像轉換成 HOG 表示，無論影像亮度如何，都可以捕捉影像的主要特徵。

為了在此 HOG 圖像中找到人臉，我們所要做的就是找到圖像中與從一堆其他訓練人臉中提取的已知 HOG 模式最相似的部分：

HOG version of our image



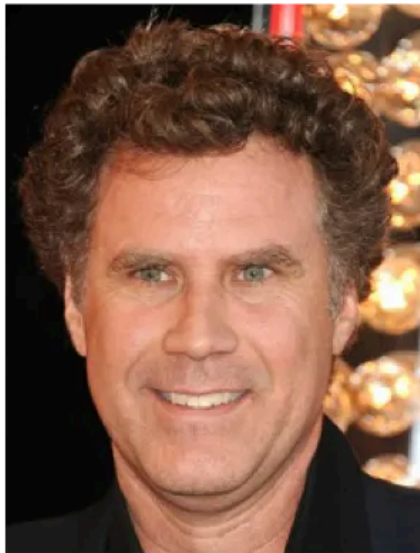
使用這種技術，我們現在可以輕鬆地在任何圖像中找到人臉：



如果您想使用 Python 和 dlib 親自嘗試此步驟，這裡有程式碼展示如何產生和檢視影像的 HOG 表示。

第二步：擺姿勢並投射臉部

呼，我們在圖像中分離出了臉部。但現在我們必須處理這樣的問題：轉向不同方向的臉在電腦看來是完全不同的：

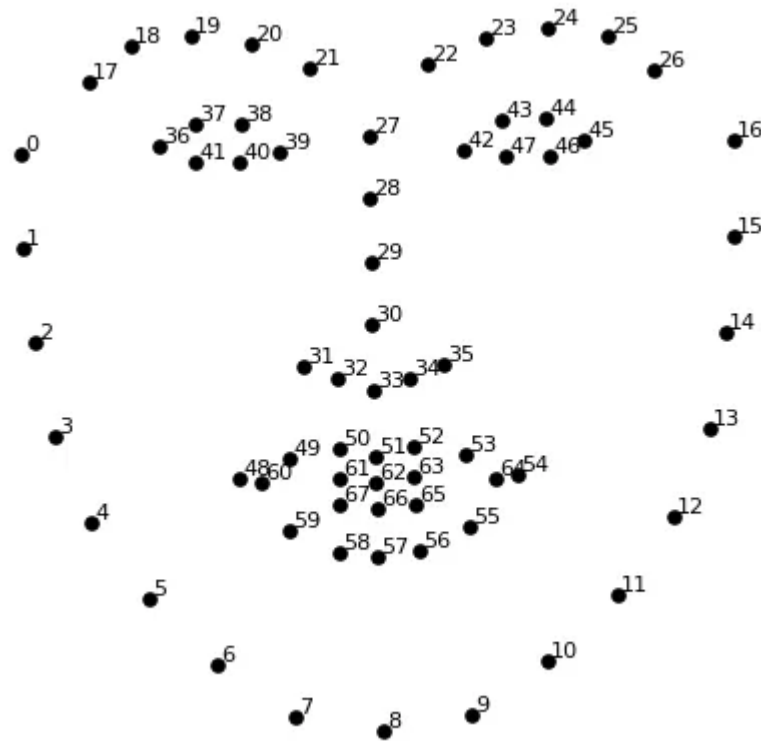


人類可以輕易辨識出這兩張照片都是威爾法洛 (Will Ferrell) 的照片，但電腦卻會將這些照片視為兩個完全不同的人。

為了解決這個問題，我們將嘗試扭曲每張圖片，以便眼睛和嘴唇始終位於圖像中的樣本位置。這將使我們在接下來的步驟中更容易比較臉。

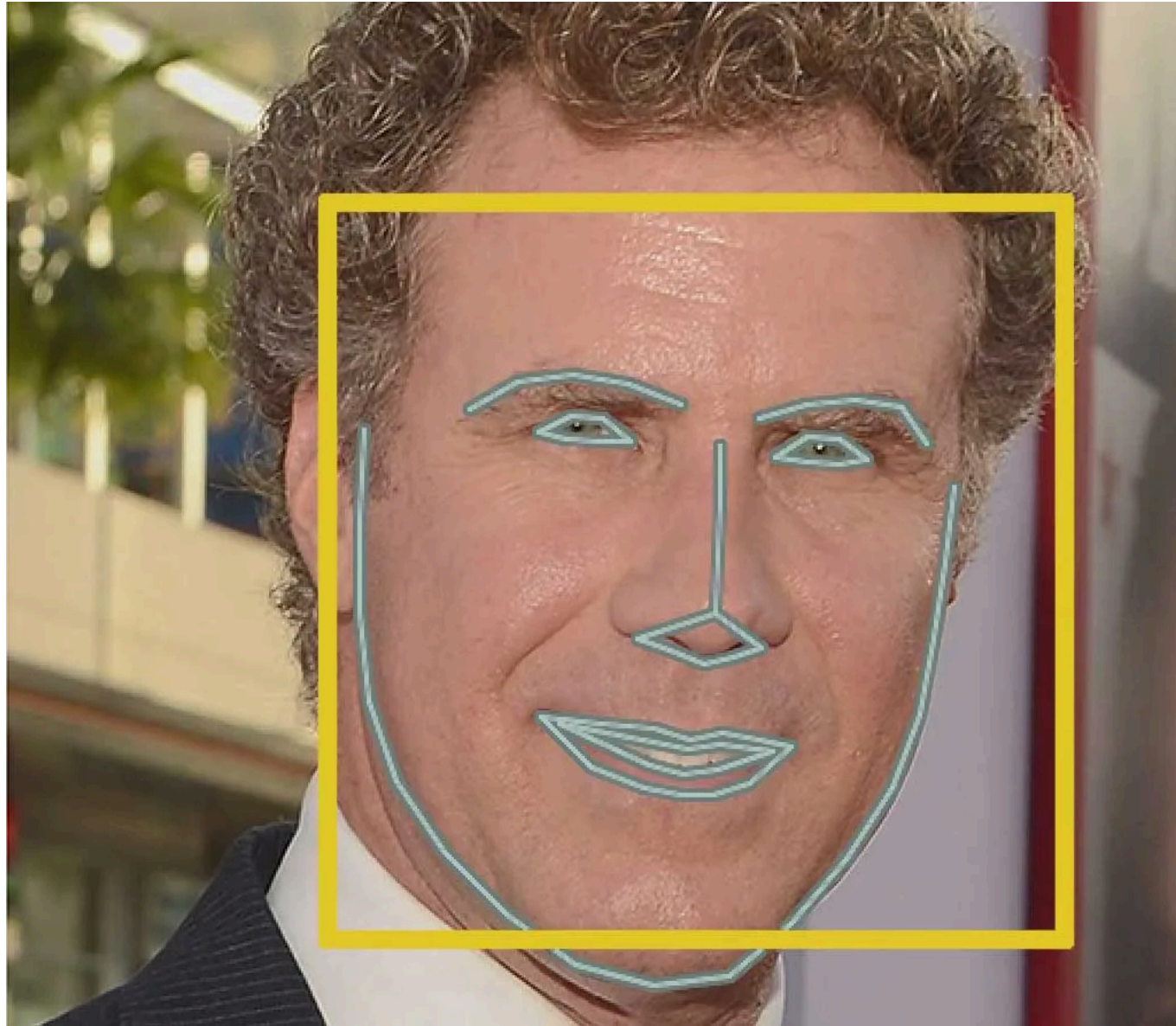
為此，我們將使用一種稱為**面部特徵估計**的演算法。有很多方法可以做到這一點，但我們將使用 Vahid Kazemi 和 Josephine Sullivan 於 2014 年發明的方法。

基本想法是，我們會找出每張臉上都存在的 68 個特定點（稱為**標誌點**），例如下巴頂部、每隻眼睛的外邊緣、每條眉毛的內邊緣等。然後，我們將訓練一個機器學習演算法，以便能夠在任何臉上找到這 68 個特定點：



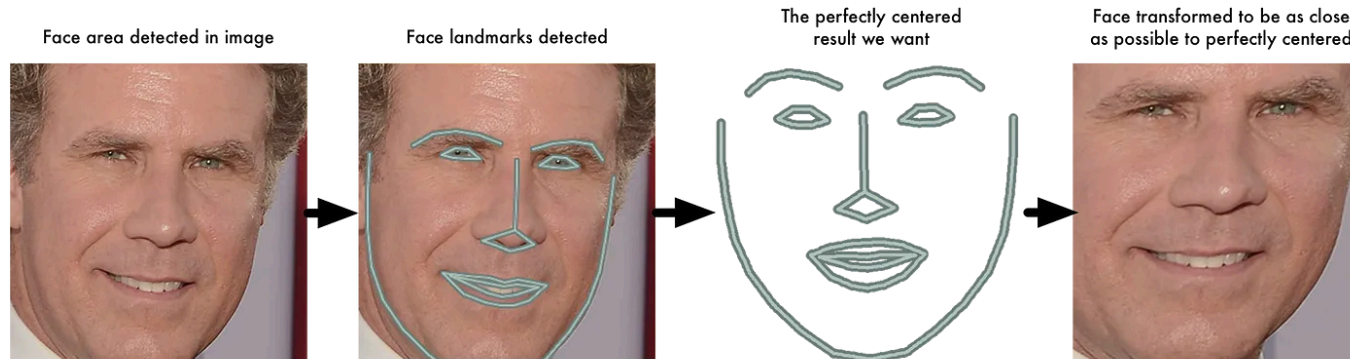
我們將在每張臉上定位 68 個地標。該圖像是由 CMU 的 [Brandon Amos](#) 創建的，他從事 [OpenFace](#) 工作。

這是我們在測試影像上定位 68 個臉部特徵點的結果：



專業提示：您也可以使用相同的技術來實現您自己的 Snapchat 即時 3D 臉部濾鏡版本！

現在我們知道了眼睛和嘴巴的位置，我們只需旋轉、縮放和剪切圖像，以使眼睛和嘴巴盡可能位於中心。我們不會做任何花哨的 3D 扭曲，因為那會導致影像扭曲。我們只使用保留平行線的基本影像變換，如旋轉和縮放（稱為仿射變換）：



現在，無論臉部如何轉動，我們都能夠將眼睛和嘴巴置於影像中大致相同的位置。這將使我們的下一步更加準確。

如果您想使用 Python 和 dlib 親自嘗試此步驟，這裡是用於查找臉部標誌的程式碼，這裡是使用這些標誌轉換圖像的程式碼。

步驟3：編碼人臉

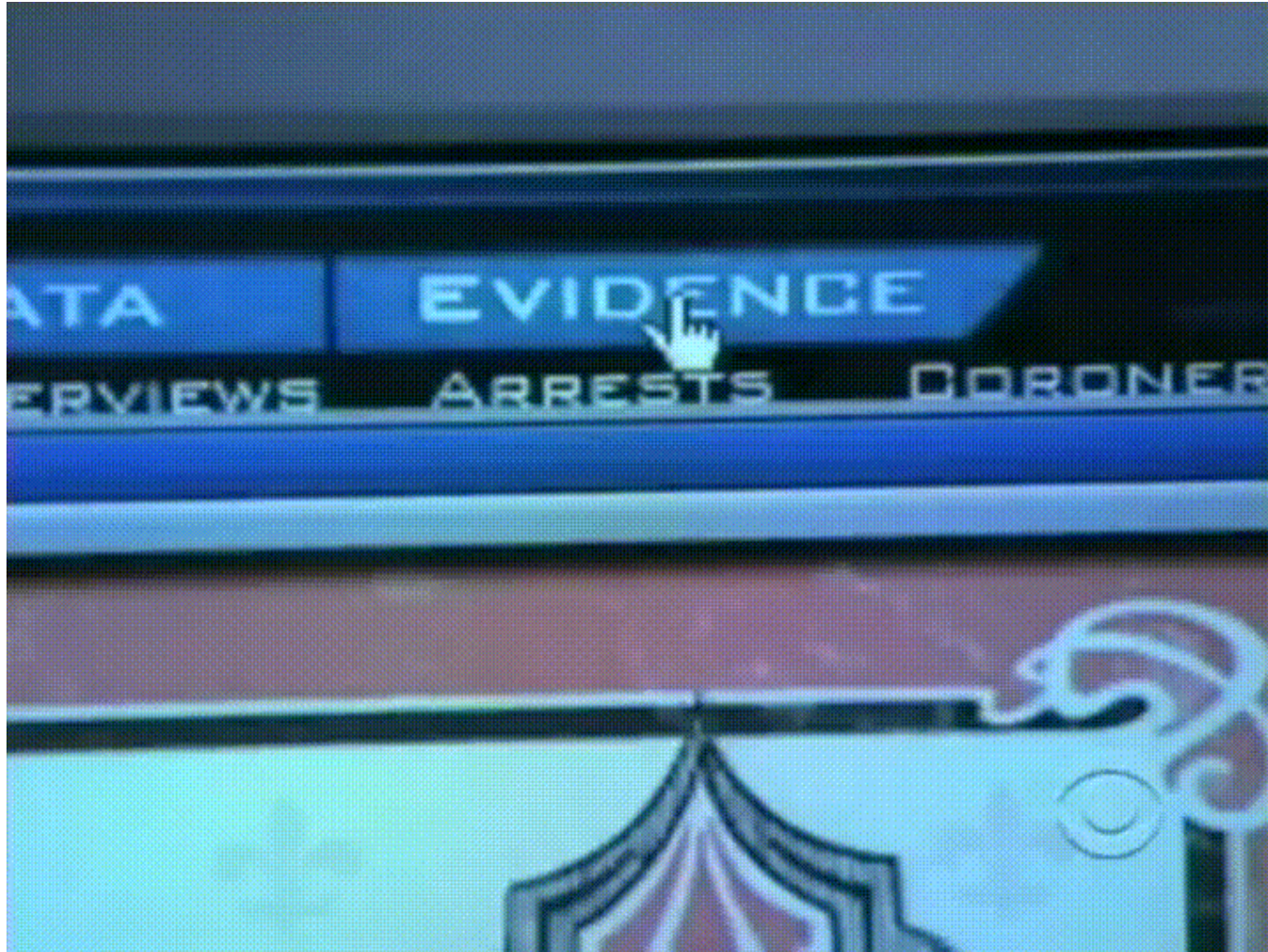
3

現在我們要討論問題的核心——如何區分臉孔。這就是事情變得非常有趣的地方！

人臉辨識最簡單的方法是直接將我們在步驟 2 中找到的未知面孔與我們擁有的所有已標記人物的照片進行比較。當我們發現先前標記過的臉孔與未知的臉孔非常相似時，那它一定是同一個人。看起來是個不錯的主意，對吧？

這種方法其實有很大的問題。像 Facebook 這樣擁有數十億用戶和數萬億張照片的網站不可能循環瀏覽每張之前標記過的臉，並將其與每張新上傳的照片進行比較。那會花太長時間。他們需要能夠在幾毫秒內而不是幾小時內辨識人臉。

我們需要一種方法來從每張臉中提取一些基本測量。然後我們可以用同樣的方法測量未知的面孔，並找到測量值最接近的已知面孔。例如，我們可能會測量每隻耳朵的大小、眼睛之間的間距、鼻子的長度等等。如果你曾經看過像《犯罪現場調查》這樣的糟糕犯罪劇，你就會明白我在說什麼：



就像電視一樣！太真實了！ #科學

測量臉部最可靠的方法

好的，那麼我們應該從每張臉部收集哪些測量值來建立已知臉部資料庫？耳朵尺寸？鼻子的長度？眼睛的顏色？還有別的嗎？

3

事實證明，對於我們人類來說顯而易見的測量值（例如眼睛顏色）對於查看影像中單一像素的電腦來說卻毫無意義。研究人員發現，最精確的方法是讓電腦自行計算要收集的測量值。深度學習在確定臉部哪些部位值得測量方面比人類做得更好。

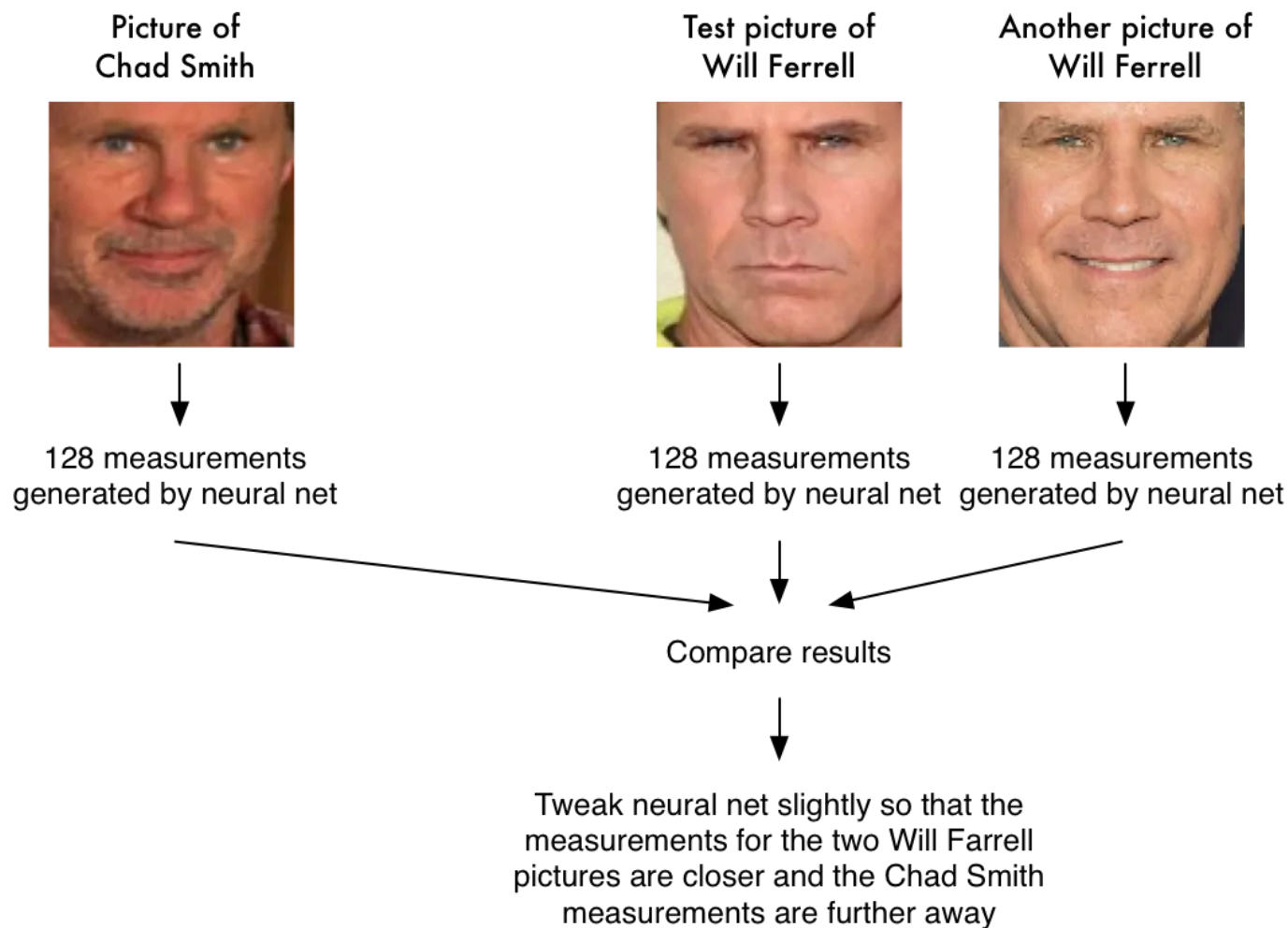
解決方案是訓練深度卷積神經網路（就像我們在第 3 部分中所做的那樣）。但我們不會像上次那樣訓練網路辨識圖片對象，而是訓練它生成每張臉 128 個測量值。

訓練過程透過一次查看 3 張臉部影像來進行：

1. 載入已知人的訓練臉部圖像
2. 載入同一已知人物的另一張照片
3. 載入一張完全不同的人的照片

然後，演算法會查看目前為這三幅影像分別產生的測量值。然後，它會稍微調整神經網絡，以確保它為 #1 和 #2 生成的測量值稍微接近，同時確保為 #2 和 #3 生成的測量值稍微遠一些：

A single 'triplet' training step:



在對數千個不同人的數百萬張圖像重複此步驟數百萬次後，神經網路學會了可靠地為每個人產生 128 個測量值。同一個人的任何十張不同照片都應給出大致相同的測量結果。

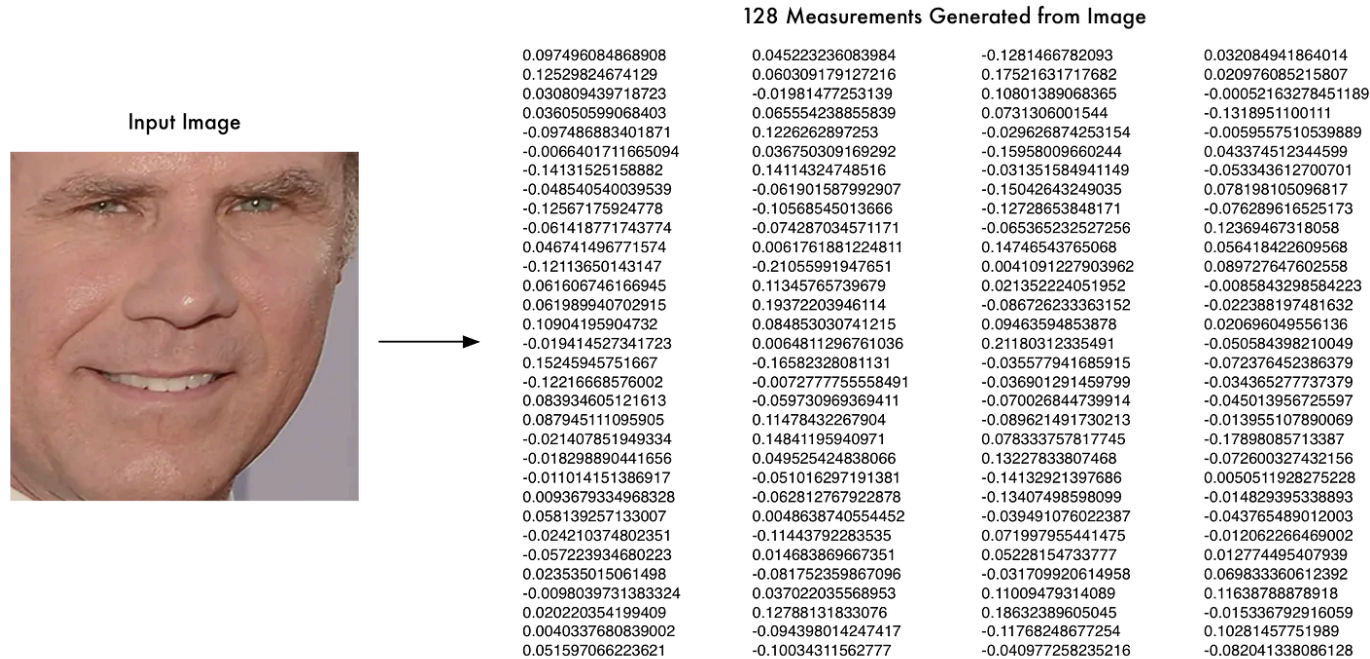
機器學習人員將每張臉的 128 個測量值稱為一個**嵌入**。將圖片等複雜的原始數據簡化為電腦生成的數字列表的想法在機器學習（尤其是在語言翻譯中）中經常出現。我們所使用的臉部辨識方法是由 Google 的研究人員在 2015 年發明的，但也有許多類似的方法。

編碼我們的臉部影像

訓練卷積神經網路輸出人臉嵌入的過程需要大量資料和電腦能力。即使使用昂貴的 Nvidia Tesla 顯示卡，也需要大約 24 小時的連續訓練才能獲得良好的準確度。

但是一旦網路經過訓練，它就可以為任何臉部產生測量值，即使是它從未見過的臉部！所以這一步只需要做一次。幸運的是，OpenFace的優秀團隊已經做到了這一點，他們發布了幾個我們可以直接使用的訓練有素的網路。感謝 Brandon Amos和他的團隊！

因此，我們需要做的就是將我們的臉部影像透過預先訓練的網路運行，以獲得每張臉部的 128 個測量值。以下是我們的測試影像的測量結果：



那麼這 128 個數字究竟測量的是臉部的哪些部位呢？事實證明我們一無所知。這對我們來說並不重要。我們所關心的是，當查看同一個人的兩張不同照片時，網路是否會產生幾乎相同的數字。

如果您想親自嘗試這一步，OpenFace 提供了一個 lua 腳本，它將生成資料夾中所有映像的嵌入並將其寫入 csv 檔案。你像這樣運行它。

步驟 4：從編碼中找出人名

這最後一步其實是整個過程中最简单的一步。我們要做的就是已知人員的資料庫中找到與我們的測試影像測量值最接近的人。

您可以使用任何基本的機器學習分類演算法來做到這一點。不需要花俏的深度學習技巧。我們將使用一個簡單的線性 SVM 分類器，但許多分類演算法都可以使用。

我們需要做的就是訓練一個分類器，它可以從新的測試圖像中獲取測量值並判斷哪個已知人是最接近的匹配。運行此分類器需要幾毫秒。分類器的結果就是這個人的名字！

那麼讓我們來試試我們的系統。首先，我用威爾法洛、查德史密斯和吉米法隆各 20 張照片的嵌入來訓練分類器：



非常棒的訓練資料！

然後，我對威爾法洛和查德史密斯在吉米法倫秀上互相假裝的著名 YouTube 影片的每一幀運行分類器：



有用！看看它對不同姿勢的臉部效果有多好——甚至是側臉！

3

自己運行

讓我們回顧一下所遵循的步驟：

1. 使用 HOG 演算法對圖片進行編碼，以建立影像的簡化版本。使用這張簡化的圖像，找到圖像中最像臉部通用 HOG 編碼的部分。
2. 透過找到臉部的主要標誌來確定臉部的姿勢。一旦我們找到這些標誌，就用它們來扭曲圖像，使眼睛和嘴巴位於中心。
3. 將居中的臉部影像傳遞給知道如何測量臉部特徵的神經網路。儲存這 128 個測量值。
4. 看看我們過去測量過的所有臉部，看看哪個人的臉部尺寸與我們的臉部尺寸最接近。這就是我們的比賽！

現在您已經了解了這一切是如何運作的，以下是如何在您自己的電腦上運行整個人臉辨識流程的從頭到尾的說明：

更新於 2017 年 4 月 9 日：您仍然可以按照以下步驟使用 OpenFace。不過，我發布了一個新的基於 Python 的人臉辨識庫，名為 [face_recognition](#)，它更容易安裝和使用。因此我建議先嘗試 [face_recognition](#)，而不是繼續下面的操作！

我甚至組裝了一個預先配置的虛擬機，其中預先安裝了 face_recognition、OpenCV、TensorFlow 和許多其他深度學習工具。您可以非常輕鬆地在電腦上下載並運行它。如果您不想自己安裝所有這些庫，請嘗試使用虛擬機器！

原始 OpenFace 說明：

Before you start

Make sure you have python, OpenFace and dlib installed. You can either [install them manually](#) or use a preconfigured docker image that has everything already installed:

```
docker pull bamos/openface
docker run -p 9000:9000 -p 8000:8000 -t -i bamos/openface /bin/bash
cd /root/openface
```

Pro-tip: If you are using Docker on OSX, you can make your OSX /Users/ folder visible inside a docker image like this:

```
docker run -v /Users:/host/Users -p 9000:9000 -p 8000:8000 -t -i bamos/openface /bin/bash
cd /root/openface
```

Then you can access all your OSX files inside of the docker image at /host/Users/...

```
ls /host/Users/
```

Step 1

Make a folder called ./training-images/ inside the openface folder.

```
mkdir training-images
```

Step 2

Make a subfolder for each person you want to recognize. For example:

```
mkdir ./training-images/will-ferrell/
mkdir ./training-images/chad-smith/
mkdir ./training-images/jimmy-fallon/
```

Step 3

Copy all your images of each person into the correct sub-folders. Make sure only one face appears in each image. There's no need to crop the image around the face. OpenFace will do that automatically.

Step 4

Run the openface scripts from inside the openface root directory:

First, do pose detection and alignment:

```
./util/align-dlib.py ./training-images/ align outerEyesAndNose ./aligned-images/ --size
```

96

This will create a new `./aligned-images/` subfolder with a cropped and aligned version of each of your test images.

Second, generate the representations from the aligned images:

```
./batch-represent/main.lua -outDir ./generated-embeddings/ -data ./aligned-images/
```

After you run this, the `./generated-embeddings/` sub-folder will contain a csv file with the embeddings for each image.

Third, train your face detection model:

```
./demos/classifier.py train ./generated-embeddings/
```

This will generate a new file called `./generated-embeddings/classifier.pkl`. This file has the SVM model you'll use to recognize new faces.

At this point, you should have a working face recognizer!

Step 5: Recognize faces!

Get a new picture with an unknown face. Pass it to the classifier script like this:

```
./demos/classifier.py infer ./generated-embeddings/classifier.pkl your_test_image.jpg
```

You should get a prediction that looks like this:

```
=== /test-images/will-ferrel-1.jpg ===
```

```
Predict will-ferrell with 0.73 confidence.
```

From here it's up to you to adapt the `./demos/classifier.py` python script to work however you want.

Important notes:

- If you get bad results, try adding a few more pictures of each person in Step 3 (especially pictures in different poses).
- This script will *always* make a prediction even if the face isn't one it knows. In a real application, you would look at the confidence score and throw away predictions with a low confidence since they are most likely wrong.

如果您喜歡這篇文章，請考慮註冊我的「機器學習很有趣！」通訊：

您也可以在 Twitter 上關注我@ageitgey，直接給我發送電子郵件或在 linkedin 上找到我。如果我能幫助您或您的團隊進行機器學習，我很樂意聽取您的意見。

現在繼續機器學習的樂趣第 5 部分！

機器學習

人工智慧

深度學習



作者：Adam Geitgey

56K 粉絲 · 37 關注

對電腦和機器學習有興趣。喜歡寫它。

跟隨

回覆 (199)

3



謝振博

您的想法是什麼？



渡邊俊介

2016年8月7日



哇！

只是.....“哇！”

這是我一生中讀過的最好的技術帖子！

這是對如此複雜的主題的實用、直觀和逐步的解釋。

你怎麼能把它拉出來？

我真的很想讀你的一篇題為「如何解釋一切看似不可思議的事情」的文章。... [更多的](#)



269



1個回覆

[回覆](#)



莫勝-G·漢尼巴爾森

2016年7月29日（已編輯）



你應該在 Udemy 上教授有關這個主題的課程或類似課程！



19

[回覆](#)

3



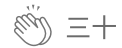
馬龍

2016年7月25日



為此，我們將影像分成每個 16x16 像素的小方塊。

對於某些影像來說，選擇 16x16 是有意義的，但是對於來自不同相機的更多樣化的影像，如何選擇這些方塊的大小呢？



三十



1個回覆

[回覆](#)[查看所有回复](#)

亞當蓋特吉的更多作品

`ram.py``data``.files``aw_data.txt`

3



亞當蓋特吉

Python 3 快速提示：在 Windows、Mac 和 Linux 上處理檔案路徑的簡...

編程的一個小煩惱是，Microsoft Windows 在資料夾名稱之間使用反斜線字符，而幾乎所...

2018年2月1日 🖱 7.5千 💬 二十七 📌 ...



亞當蓋特吉

自然語言處理很有趣！

電腦如何理解人類語言

2018年7月18日 🖱 22千 💬 64 📌 ...

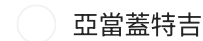


亞當蓋特吉

機器學習很有趣！

世界上最簡單的機器學習入門書

2014年5月6日 🖱 9萬 💬 265 📌 ...



亞當蓋特吉

機器學習很有趣！第三部分：深度學習與卷積神經網絡

更新：本文是系列文章的一部分。看完整系列：第1部分、第2部分、第3部分、第4...

2016年6月14日 🖱 26千 💬 191 📌 ...

Medium 推薦

 在 編碼之美 經過 塔里·伊巴巴

Google 的這款新 IDE 絕對會改變遊戲規則

谷歌的這個新 IDE 確實具有革命性。

🌟 3月12日 🖱️ 5.6千 💬 335  ...

 在 邁向人工智慧 經過 鮑里斯·梅納德斯

2025 年我將如何學習機器學習（如果可以重新開始的話）

2025 年，學習 ML 所需的只是一台筆記型電腦和一份必須採取的步驟清單。

🌟 1月2日 🖱️ 2.2千 💬 72  ...

 在 LearnAlforproft.com 經過 尼普納·馬杜蘭加

無需辭職，即可利用人工智慧賺錢

我每天做 2 小時

★ 3月25日 🖱️ 10.4 千 💬 460 📌⁺ ⋮

 德夫拉傑·阿加瓦爾

使用 MTCNN、FaceNet 和 Milvus 建立人臉辨識系統

介紹

1月8日 🖱️ 5 📌⁺ ⋮

 在 數據科學集體 經過 保羅·佩羅內

使用 LangGraph 建立您的第一個 AI 代理程式的完整指南。（比你想像...

 阿什哈達利

使用 OpenCV 進行影像處理—影像和影片的人臉偵測

在建立我的第一個商業 AI 代理三個月後，一切都在客戶演示期間崩潰了。

在本節中，我們將探討如何使用 OpenCV 偵測影像中的人臉。為此，我們將利用預先訓練...

 3月12日

 3.5千

 69

2024 年 11 月 24 日

看更多推薦